



1.- CAPÍTULO I: 1. Arreglos, listas y archivos

Programación en R

Abril 2026 - Agosto 2026

Ciencia de Datos & Inteligencia Artificial

1. Programación en R

1.1 Introducción

1. Programación en R

1.1 Introducción

- R es un lenguaje y entorno para computación estadística.
- Software libre bajo licencia GNU.
- Amplia variedad de técnicas estadísticas y matemáticas.
- Potente capacidad gráfica y de visualización.

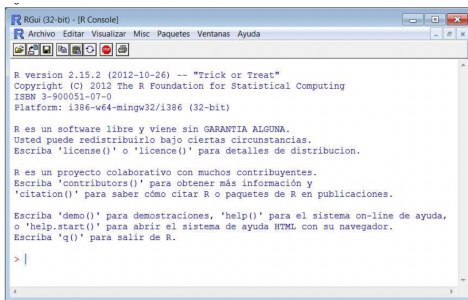
- Instalar y configurar R y R-Commander.
- Comprender la sintaxis básica de R.
- Manejar vectores, matrices y listas.

- Lenguaje de programación orientado al análisis de datos.
- Utilizado en investigación y docencia.
- Incluye librerías estadísticas y matemáticas.
- Permite crear funciones y algoritmos propios.

- Gratuito y de código abierto.
- Gran comunidad de usuarios.
- Miles de paquetes disponibles.
- Capacidad de automatización y programación.

- Curva de aprendizaje elevada.
- Basado inicialmente en consola.
- Requiere conocimientos básicos de programación.

- Descargar desde: www.r-project.org
- Seleccionar versión según el sistema operativo.
- Instalar distribución base.
- Mantener actualizado el software.



```
RGui (32-bit) - [R Console]
R Archivo Editar Visualizar Misc Paquetes Ventanas Ayuda

R version 2.15.2 (2012-10-26) -- "Trick or Treat"
Copyright (C) 2012 The R Foundation for Statistical Computing
ISBN 3-900051-07-0
Platform: i386-w64-mingw32/i386 (32-bit)

R es un software libre y viene sin GARANTIA ALGUNA.
Usted puede redistribuirlo bajo ciertas circunstancias.
Escriba 'license()' o 'licence()' para detalles de distribución.

R es un proyecto colaborativo con muchos contribuyentes.
Escriba 'contributors()' para obtener más información y
'citation()' para saber cómo citar R o paquetes de R en publicaciones.

Escriba 'demo()' para demostraciones, 'help()' para el sistema on-line de ayuda,
o 'help.start()' para abrir el sistema de ayuda HTML con su navegador.
Escriba 'q()' para salir de R.

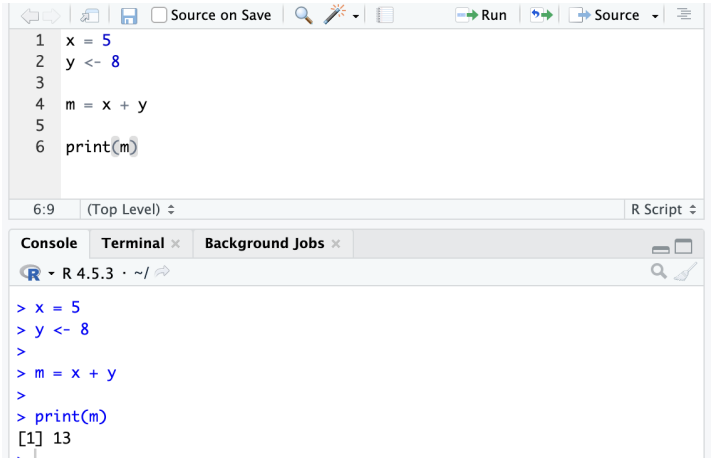
> |
```

- Uso de la consola de comandos.
- Ejemplo:

Ejemplo

```
> 2 * 3  
[1] 6
```

- Uso de scripts para análisis extensos.



The screenshot displays the RStudio interface. The top toolbar includes icons for navigation, saving, and running code. The source editor shows a script with the following R code:

```
1 x = 5
2 y <- 8
3
4 m = x + y
5
6 print(m)
```

Below the script editor, the console window is active, showing the execution of the code:

```
> x = 5
> y <- 8
>
> m = x + y
>
> print(m)
[1] 13
```

- Los paquetes amplían funcionalidades.
- Instalación:

Código

```
install.packages('paquete')
```

- Carga de paquetes:

Código

```
library(paquete)
```

- Numéricos.
- Caracteres.
- Lógicos (TRUE/FALSE).
- Estructuras: vectores, matrices, listas y data frames.

Estructuras:

Objeto	Descripción
Data frame	Estructuras de datos bidimensionales, donde se recogen diferentes variables por columnas.
Vector	Colección ordenada de datos con una longitud determinada.
Array	Conjunto de datos indexado por dos o más índices, caracterizado por tener dimensión.
Matrix	Caso particular de <i>array</i> cuando hay dos índices, es decir, dos dimensiones (filas y columnas).
List	Objeto que recoge varios tipos de elementos (componentes), que pueden ser de clases diferentes.
Function	Pieza de código que puede ser un algoritmo, una función matemática o una estructura lógica.
Factor	Vector que especifica una clasificación discreta de los elementos de otro vector de la misma longitud.

- Estructura básica de almacenamiento.
- Creación con `c()`.

Ejemplo

```
a <- c(1,2,3,4)
```

- Operaciones vectorizadas.

- Estructura básica de almacenamiento.
- Creación con `c()`.

Ejemplo

```
a <- c(1,2,3,4)
```

- Operaciones vectorizadas.

```
> a <- c(1,2,3,4,5,6)
```

```
> a
```

```
> a <- c(1,2,3)
```

```
> b <- c(7,8,9)
```

```
> ab <- c(a,4,5,6,b)
```

```
> ab
```

```
[1] 1 2 3 4 5 6 7 8 9
```

```
> |
```

```
v1 <- c(1, 2, 3)
```

```
v2 <- c(2, 4, 6)
```

Descripción	Instrucciones	Resultado
Concatenación	<code>c(v1, v2)</code>	1, 2, 3, 4, 5, 6
Suma	<code>v1+v2</code>	3, 6, 9
Resta	<code>v1-v2</code>	-1, -2, -3
Multiplicación	<code>v1*v2</code>	2, 8, 18
División	<code>v1/v2</code>	0.5, 0.5, 0.5
Potencia	<code>v1^v2</code>	1, 16, 729

```
v3 <- c(-1, 0, 1, 2)
```

Descripción	Instrucciones	Resultado
Raíz cuadrada	<code>sqrt(v3)</code>	NaN 0.00 1.00 1.41
Valor absoluto	<code>abs(v3)</code>	1 0 1 2
Exponencial	<code>exp(v3)</code>	0.36 1.00 2.71 7.38
Log. base e	<code>log(v3)</code>	NaN -Inf 0.00 0.69
Log. base 10	<code>log10(v3)</code>	NaN -Inf 0.00 0.30
Factorial (!)	<code>factorial(v3)</code>	NaN 1 1 2

```
v4 <- c(-2, 3, 5, 0, 0, 3)
```

```
v5 <- c(20, 14, 51, 76, 21, 30)
```

Descripción	Instrucciones	Resultado
Longitud	<code>length(v4)</code>	6
Máximo	<code>max(v4)</code>	5
Mínimo	<code>min(v4)</code>	-2
Suma	<code>sum(v4)</code>	9
Producto	<code>prod(v4)</code>	0
Media	<code>mean(v4)</code>	1.5
Mediana	<code>median(v4)</code>	1.5
Desviación estándar	<code>sd(v4)</code>	2.58

- Suma, resta y multiplicación.
- Potencias y raíces.
- Funciones estadísticas:
 - `mean()`
 - `median()`
 - `sd()`
 - `var()`

- Creación con `seq()` y `:`
- Repeticiones con `rep()`
- Inversión con `rev()`

```
v6 <- c(1,2,3)
```

Instrucciones	Resultado
<code>rep(v6,2)</code>	1 2 3 1 2 3
<code>rep(v6,each=2)</code>	1 1 2 2 3 3
<code>rep(v6,each=2,times=2)</code>	1 1 2 2 3 3 1 1 2 2 3 3
<code>rep(v6,c(2,3,4))</code>	1 1 2 2 2 3 3 3 3
<code>1:10</code>	1 2 3 4 5 6 7 8 9 10
<code>seq(1,10)</code>	1 2 3 4 5 6 7 8 9 10
<code>10:1</code>	10 9 8 7 6 5 4 3 2 1
<code>seq(10,1)</code>	10 9 8 7 6 5 4 3 2 1
<code>seq(4,length=8)</code>	4 5 6 7 8 9 10 11
<code>seq(0,40,by=5)</code>	0 5 10 15 20 25 30 35 40
<code>seq(0,1,length=4)</code>	0.00 0.33 0.66 1.00
<code>seq(from=2,to=8,by=3)</code>	2 5 8
<code>rev(1:5)</code>	5 4 3 2 1
<code>2*1:4</code>	2 4 6 8

- Comparaciones: $<$, $>$, $==$, $!=$
- Operadores lógicos:
 - $\&$ (y)
 - $|$ (o)
 - $!$ (negación)

```
> a <- c(-2,-1,0,1,2)
> a >2
[1] FALSE FALSE FALSE FALSE FALSE
> a <= -1
[1] TRUE TRUE FALSE FALSE FALSE
> a >0 & abs(a) == 2
[1] FALSE FALSE FALSE FALSE TRUE
> a >0 | abs(a) == 2
[1] TRUE FALSE FALSE TRUE TRUE
```

`v7 <- c(-3, -2, -1, 0, 1, 2, 3)` o de forma más compacta `v7 <- c(-3:3)`

Expresión	Resultado	Descripción
<code>v7[1]</code>	-3	Primer elemento
<code>v7[2:4]</code>	-2 -1 0	Del segundo al cuarto elemento
<code>v7[-(2:length(v7))]</code>	-3	Todos menos aquellos desde el segundo hasta el último elemento
<code>v7[c(2, 4, 6)]</code>	-2 0 2	Elementos segundo, cuarto y sexto
<code>v7[v7>0]</code>	1 2 3	Elementos mayores que 0
<code>v7[v7!=0]</code>	-3 -2 -1 1 2 3	Elementos diferentes de 0
<code>v7[abs(v7)==3]</code>	-3 3	Elementos con valor absoluto 3
<code>v7[v7>0 & v7!=2]</code>	1 3	Elementos positivos y que no sean 2
<code>v7[v7<0 v7==3]</code>	-3 -2 -1 3	Elementos negativos o que sean 3

- Estructuras bidimensionales.
- Creación con `matrix()`.
- Operaciones matriciales.
- Funciones: `det()`, `solve()`, `t()`.

- Creación de funciones personalizadas.
- Sintaxis:

Sintaxis

```
nombre <- function(argumentos){ }
```

- Facilitan reutilización de código.

```
11 ▾ suma <- function (x, y){  
12     return (x+y)  
13 ▲ }  
14  
15 r <- suma(4,7)  
16 print(r)  
17  
18  
19 ▾ producto <- function (x, y){  
20     return (x*y)  
21 ▲ }  
22  
23 r <- producto(4,7)  
24 print(r)
```

- Uso de `ifelse()`.
- Ciclos `for`.
- Automatización de procesos iterativos.

```
edad <- 16

if (edad >= 18) {
  print("Mayor de edad")
} else {
  print("Menor de edad")
}
```

```
nota <- 75

if (nota >= 90) {
  print("Excelente")
} else if (nota >= 70) {
  print("Aprobado")
} else {
  print("Reprobado")
}
```

```
for (i in 1:5) {  
    print(i^2)  
}
```

```
nombres <- c("Ana", "Luis", "Carlos")  
  
for (nombre in nombres) {  
  print(nombre)  
}
```

```
suma <- 0

for (i in 1:5) {
  suma <- suma + i
}

print(suma)
```

Crear una función que imprima la tabla de multiplicar de un número.

Crear una función que imprima las tabla de multiplicar, hasta el número.

Crear una función que calcule a^b usando multiplicaciones repetidas con for.

Crear una función que cuente cuántos números pares existen entre 1 y n .

Ejercicios 5

Crear una función que cuente cuántos números impares existen entre 1 y n.

Ejercicios 6

Crear una función que sume solo los números pares entre 1 y n.

Ejercicios 7

Crear una función que reciba un vector y calcule el promedio usando un for.

Ejercicios 8

Crear una función que encuentre el número mayor de un vector usando for.

Ejercicios 9

Crear una función que invierta un vector usando for.

Ejercicios 10

Crear una función que cuente cuántas vocales tiene una palabra.

Familia apply

- `apply()` para filas y columnas.
- `tapply()` para grupos o factores.
- Alternativa eficiente a ciclos.

- Uso de `data.frame()`.
- Manejo de variables y observaciones.
- Filtrado mediante `subset()`.

- Permiten almacenar objetos diferentes.
- Acceso mediante \$ y `[[ ]]`
- Muy útiles para análisis complejos.

- Interfaz gráfica para R.
- Basada en menús desplegables.
- Facilita el aprendizaje inicial.
- Muestra el código generado.

Instalación de R-Commander

Código

```
install.packages('Rcmdr')
```

Carga del paquete

```
library(Rcmdr)
```

Conclusiones

- R es una herramienta poderosa para análisis estadístico.
- Combina programación y análisis cuantitativo.
- R-Commander facilita el aprendizaje.
- Fundamental para ciencia de datos y estadística.